TransitOps — Smart Transport Operations Platform

Complete End-to-End Build Document (8-Hour Hackathon)

---

1. Project Summary

TransitOps digitizes transport operations — vehicles, drivers, trips, maintenance, fuel/expenses, and analytics — replacing spreadsheets with a rule-enforced, role-based web platform.

Core value delivered: no double-booked vehicles/drivers, no overloaded cargo, no expired-license dispatch, automatic status lifecycle, real-time cost/utilization visibility.

---

2. Tech Stack

Layer	Choice	Reasoning
Frontend	React + Vite + TailwindCSS + shadcn/ui	Fast scaffolding, prebuilt table/form/dashboard components
Charts	Recharts	Quick KPI and analytics visuals
Backend	Node.js + Express (or FastAPI/Python)	Simple REST, fast to wire up in hours
Database	PostgreSQL	Relational integrity, enums, unique/FK constraints
ORM	Prisma (Node) / SQLAlchemy (Python)	Fast schema iteration + migrations
Auth	JWT + bcrypt	Stateless, simple to implement RBAC on top of
State Mgmt	React Query / Zustand	Server-state caching, auto-refresh of KPIs
Deployment	Render/Railway (backend) + Vercel (frontend)	Zero-config deploy for live demo

Hackathon constraint: single monorepo, single Postgres instance, one backend service — no microservices, no message queues.

---

3. Database Schema (DDL)

```sql
```

```
CREATE TYPE user_role AS ENUM ('FleetManager','Driver','SafetyOfficer','Financi
CREATE TYPE vehicle_status AS ENUM ('Available','On Trip','In Shop','Retired');
CREATE TYPE driver_status AS ENUM ('Available','On Trip','Off Duty','Suspended'
CREATE TYPE trip_status AS ENUM ('Draft','Dispatched','Completed','Cancelled');
CREATE TYPE maintenance_status AS ENUM ('Open','Closed');
CREATE TYPE expense_category AS ENUM ('Toll','Maintenance','Other');

CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  email TEXT UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  role user_role NOT NULL,
  created_at TIMESTAMP DEFAULT now()
);

CREATE TABLE vehicles (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  reg_number TEXT UNIQUE NOT NULL,
  name TEXT NOT NULL,
  type TEXT NOT NULL,
  max_load_kg NUMERIC NOT NULL CHECK (max_load_kg > 0),
  odometer NUMERIC DEFAULT 0,
  acquisition_cost NUMERIC NOT NULL,
  status vehicle_status DEFAULT 'Available',
  region TEXT,
  created_at TIMESTAMP DEFAULT now()
);

CREATE TABLE drivers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  license_number TEXT UNIQUE NOT NULL,
  license_category TEXT NOT NULL,
  license_expiry DATE NOT NULL,
  contact_number TEXT,
  safety_score NUMERIC DEFAULT 100,
  status driver_status DEFAULT 'Available',
  created_at TIMESTAMP DEFAULT now()
);

CREATE TABLE trips (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  source TEXT NOT NULL,
  destination TEXT NOT NULL,
  vehicle_id UUID NOT NULL REFERENCES vehicles(id),
```

```
  driver_id UUID NOT NULL REFERENCES drivers(id),
  cargo_weight NUMERIC NOT NULL,
  planned_distance NUMERIC NOT NULL,
  actual_distance NUMERIC,
  revenue NUMERIC DEFAULT 0,
  status trip_status DEFAULT 'Draft',
  created_at TIMESTAMP DEFAULT now(),
  dispatched_at TIMESTAMP,
  completed_at TIMESTAMP
);

CREATE TABLE maintenance_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  vehicle_id UUID NOT NULL REFERENCES vehicles(id),
  type TEXT NOT NULL,
  description TEXT,
  cost NUMERIC DEFAULT 0,
  status maintenance_status DEFAULT 'Open',
  created_at TIMESTAMP DEFAULT now(),
  closed_at TIMESTAMP
);

CREATE TABLE fuel_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  vehicle_id UUID NOT NULL REFERENCES vehicles(id),
  trip_id UUID REFERENCES trips(id),
  liters NUMERIC NOT NULL,
  cost NUMERIC NOT NULL,
  log_date DATE NOT NULL
);

CREATE TABLE expenses (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  vehicle_id UUID NOT NULL REFERENCES vehicles(id),
  category expense_category NOT NULL,
  amount NUMERIC NOT NULL,
  expense_date DATE NOT NULL,
  notes TEXT
);
```

Note on ROI formula: the spec defines `ROI = (Revenue − (Maintenance + Fuel)) / Acquisition Cost`, but doesn't define where "Revenue" comes from. Recommended assumption (state this explicitly in your demo): add a `revenue` field on `trips`, either entered manually per trip or computed as `distance × rate_per_km`. Document this assumption on a slide — judges will ask.

4. API Design

```
POST   /api/auth/login
POST   /api/auth/register            (FleetManager/SafetyOfficer only, per
RBAC)

GET    /api/vehicles                 ?status=&type=&region=
POST   /api/vehicles
PATCH  /api/vehicles/:id
GET    /api/vehicles/available       (pre-filtered for dispatch dropdown)

GET    /api/drivers                  ?status=
POST   /api/drivers
PATCH  /api/drivers/:id
GET    /api/drivers/available        (pre-filtered: not Suspended, license
not expired, not On Trip)

POST   /api/trips                    (create Draft)
POST   /api/trips/:id/dispatch       (validates + transitions vehicle/driver
→ On Trip)
POST   /api/trips/:id/complete       (captures odometer/fuel, transitions →
Available)
POST   /api/trips/:id/cancel         (reverts vehicle/driver → Available)
GET    /api/trips                    ?status=

POST   /api/maintenance              (auto-sets vehicle → In Shop)
POST   /api/maintenance/:id/close    (auto-sets vehicle → Available, unless
Retired)

POST   /api/fuel-logs
POST   /api/expenses

GET    /api/dashboard/kpis
GET    /api/reports/fuel-efficiency
GET    /api/reports/utilization
GET    /api/reports/operational-cost
GET    /api/reports/roi
GET    /api/reports/export.csv
```

5. Business Rule Enforcement (Domain Layer)

Centralize all rules in a `TripService` / `MaintenanceService` — not in controllers — since

these are graded criteria and need to be demonstrably correct and testable.

`TripService.dispatch(tripId)` — runs inside a single DB transaction:

1. Load trip, vehicle, driver.
2. Reject if `vehicle.status != 'Available'`.
3. Reject if `driver.status != 'Available'`.
4. Reject if `driver.license_expiry < today`.
5. Reject if `driver.status == 'Suspended'`.
6. Reject if `trip.cargo_weight > vehicle.max_load_kg`.
7. On success: `trip.status = Dispatched`, `vehicle.status = On Trip`, `driver.status = On Trip`, `trip.dispatched_at = now()`.

`TripService.complete(tripId, finalOdometer, fuelConsumed)`

1. `trip.status = Completed`, `vehicle.status = Available`, `driver.status = Available`.
2. `vehicle.odometer = finalOdometer`.
3. Optionally auto-insert a `fuel_logs` row from `fuelConsumed`.

`TripService.cancel(tripId)`

1. Only valid from `Dispatched` state.
2. `trip.status = Cancelled`, `vehicle.status = Available`, `driver.status = Available`.

`MaintenanceService.create(vehicleId, ...)`

1. Insert record with `status = Open`.
2. `vehicle.status = In Shop` immediately (removes from dispatch pool).

`MaintenanceService.close(logId)`

1. `maintenance.status = Closed`, `closed_at = now()`.
2. `vehicle.status = Available` **unless** `vehicle.status == Retired`.

Critical implementation rule: never trust frontend filtering alone. The dropdown pre-filters for UX, but the backend must re-validate every rule at the moment of dispatch to prevent race conditions (e.g., two dispatchers submitting near-simultaneously).

Action	Fleet Manager	Driver	Safety Officer	Financial Analyst
CRUD Vehicles	✅	❌	❌	View only
CRUD Drivers	View only	❌	✅	View only
Create/Dispatch/Complete Trips	✅	✅	❌	❌
Maintenance Logs	✅	❌	❌	View only
Fuel/Expense Entry	✅	✅ (own trips)	❌	View only
Reports & Analytics	✅	Own trips only	Compliance view	✅ Full access
License/Safety Compliance	View only	❌	✅	❌

Implement as Express middleware: `requireRole(['FleetManager', 'SafetyOfficer'])` per route.

---

7. End-to-End Workflow

A. Onboarding

1. Fleet Manager registers a vehicle (unique `reg_number` enforced at DB + API level) → status `Available`.
2. Safety Officer registers a driver with license expiry date → status `Available`.

B. Trip Creation & Dispatch 3. Driver opens "Create Trip" → vehicle/driver dropdowns call `/available` endpoints (excludes Retired/In Shop/On Trip vehicles; excludes Suspended/expired-license/On Trip drivers). 4. Enter source, destination, cargo weight, planned distance → submit → validated against `max_load_kg` → saved as `Draft`. 5. Click "Dispatch" → `TripService.dispatch()` runs full validation chain → on success, trip → `Dispatched`, vehicle & driver → `On Trip`.

C. Completion 6. Driver/Fleet Manager clicks "Complete Trip" → enters final odometer + fuel consumed → trip → `Completed`, vehicle & driver → `Available`, vehicle odometer updated.

D. Cancellation 7. A `Dispatched` trip can be cancelled → reverts vehicle & driver to `Available`.

E. Maintenance 8. Fleet Manager creates a maintenance record for a vehicle → vehicle instantly → `In Shop`, excluded from dispatch pool. 9. Closing the record → vehicle → `Available` (unless separately marked `Retired`).

F. Fuel & Expenses 10. Fuel logs (liters, cost, date) and expenses (tolls, misc) recorded against a vehicle. 11. Backend aggregates `total_operational_cost = SUM(fuel.cost) + SUM(maintenance.cost)` per vehicle.

G. Reporting 12. Dashboard/report endpoints compute Fuel Efficiency, Fleet Utilization %, Operational Cost, and ROI; CSV export available; PDF export optional (bonus).

Validated example (from spec): Van-05 (500kg capacity) + Driver Alex → 450kg trip → dispatch allowed (450 ⩽ 500) → both go `On Trip` → complete → both `Available` → Oil Change maintenance record → vehicle `In Shop` → reports refresh.

---

8. 8-Hour Execution Timeline (suggested team split: 4 people)

Time	Backend Dev	Frontend Dev	Full-stack/Integration	Design/QA
Hr 0–1	Repo setup, DB schema, migrations	Project scaffold, routing, auth pages	Environment/deploy pipeline setup	Review mockup, finalize UI flow
Hr 1–2.5	Auth + RBAC middleware, Vehicle/Driver CRUD APIs	Login page, Vehicle Registry UI, Driver Mgmt UI	Wire auth to frontend	Component styling (Tailwind/shadcn)
Hr 2.5–4.5	Trip lifecycle APIs + business rule engine	Trip creation form w/ filtered dropdowns, trip list	Wire trip flows end-to-end	Test dispatch validation edge cases
Hr 4.5–6	Maintenance + Fuel/Expense APIs, cost aggregation	Maintenance UI, Fuel/Expense forms	Wire maintenance + fuel flows	Test status transitions
Hr 6–7	Dashboard/report endpoints, CSV export	Dashboard KPIs, charts (Recharts)	Wire dashboard, filters	Cross-role testing (RBAC)

Time	Backend Dev	Frontend Dev	Full-stack/Integration	Design/QA
Hr 7–8	Bug fixes, seed data	Responsive polish, dark mode (if time)	Final deploy, demo script	Full workflow demo run-through

---

9. Mandatory Deliverables Checklist

- [ ] Responsive web interface
- [ ] Authentication with RBAC (4 roles)
- [ ] CRUD for Vehicles and Drivers
- [ ] Trip Management with full validation chain
- [ ] Automatic status transitions (vehicle/driver/trip/maintenance)
- [ ] Maintenance workflow (Open → In Shop → Closed → Available)
- [ ] Fuel & Expense tracking with cost aggregation
- [ ] Dashboard with KPIs (Active/Available Vehicles, In Maintenance, Active/Pending Trips, Drivers On Duty, Fleet Utilization %)
- [ ] Charts and visual analytics
- [ ] CSV export

10. Bonus Features (time-permitting, priority order)

1. Email reminders for expiring licenses (cron/scheduled job — high judge visibility, low effort)
2. Search, filters, and sorting (likely already partially built via KPI filters)
3. Dark mode (Tailwind class toggle — cheap if using shadcn/ui)
4. Vehicle document management (file upload for RC/insurance docs)
5. PDF export (marked optional even for core reports)

---

11. Key Risks & Assumptions to State in Demo

- **Revenue field is undefined in the spec** — assumption made (see Section 3) for ROI calculation; call this out explicitly to judges rather than letting it look like an oversight.
- **Race conditions on dispatch** — mitigated via DB transactions; worth a one-line mention in the demo to show engineering rigor.
- **Mockup/video reference:** the provided Excalidraw mockup link requires an interactive session and could not be rendered for this document; the provided video appears to be a narrated briefing rather than a UI walkthrough. Confirm exact screen layouts against the live mockup before final UI build.